

TRƯỜNG ĐẠI HỌC CÔNG NGHIỆP TP.HCM

Khoa Công Nghệ Điện Tử - Bộ môn Điện Tử Máy Tính

BÀI GIẢNG

**HỆ THỐNG NHÚNG
(Embedded System)**

TP.HCM-2024

NỘI DUNG MÔN HỌC

Chương 1: GIỚI THIỆU HỆ THỐNG NHÚNG

Chương 2: KỸ THUẬT GIAO TIẾP NGOẠI VI

Chương 3: KỸ THUẬT PHẦN MỀM

Chương 4: THIẾT KẾ HỆ THỐNG NHÚNG

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

1. NGÔN NGỮ LẬP TRÌNH
2. **CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG**
3. TÍNH TOÁN THỜI GIAN VÀ TỐI ƯU

Contents

- ☐ Choosing the best software architecture
- ☐ Basic real-time software architecture
- ☐ Real Time Operating System

2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG

Contents

- ❑ Choosing the best software architecture
- ❑ Basic real-time software architecture
- ❑ Real Time Operating System

2.1 Choosing the best software architecture

❑ **The best architecture** depends on several **factors**:

- Real-time requirements of the **application** (absolute response time).
- Available **hardware** (speed, features).
- Number and complexity of **different software features**.
- Number and complexity of **different peripherals**.
- Corresponding priority of features.

2.1 Choosing the best software architecture

- ❑ The decision is based on the tradeoff between **complexity** and **control** over response and priority:
 - Systems that require **little control** and **poor response** can be done with **simple architectures**.
 - **Rapid response systems** will require **more complex program design** to be successful.

2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG

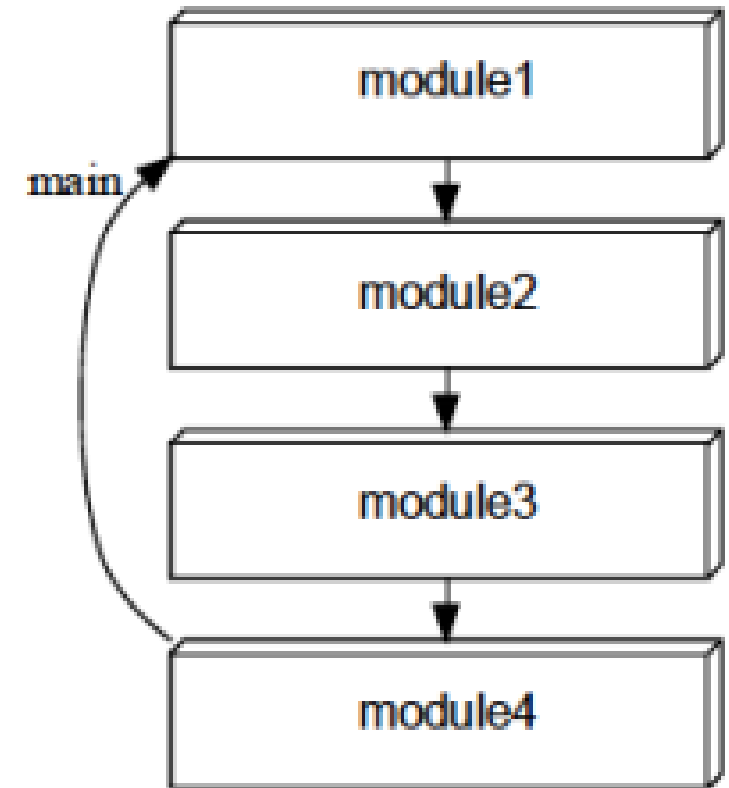
Contents

- ☐ Choosing the best software architecture
- ☐ **Basic real-time software architecture**
- ☐ Real Time Operating System

2.2 Basic real-time software architecture

1). Round Robin

- + Simplest architecture.
- + No interrupts.
- + Main loop checks each device one **at a time**, and service whichever needs to be serviced.
- + **Service order** depends on position in the loop.



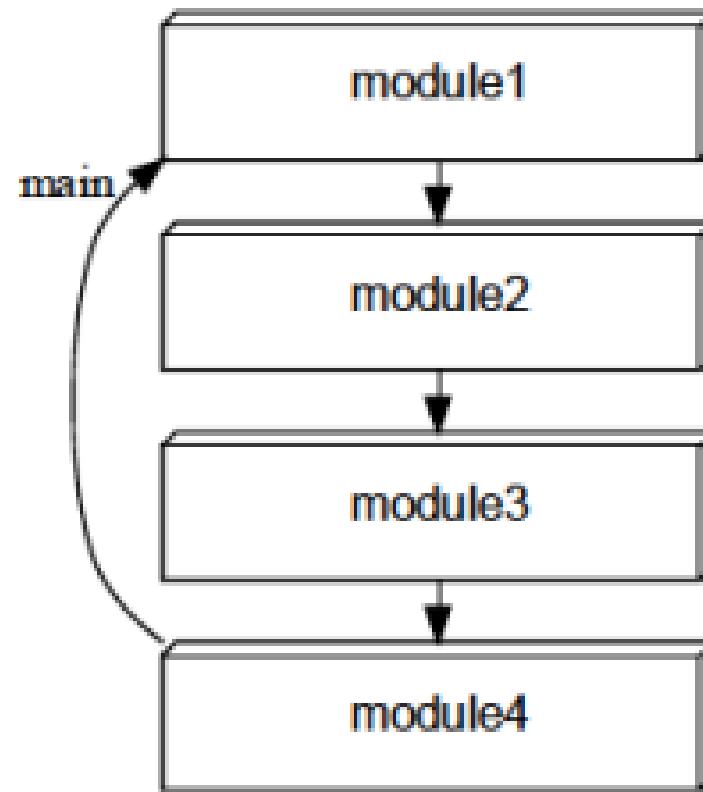
2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

2.2 Basic real-time software architecture

❑ Round Robin

- + No priorities.
- + No shared data.



```
void main()  
{ while (TRUE)  
    { module1();  
      module2();  
      module3();  
      module4();  
    }  
}
```

2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

2.2 Basic real-time software architecture

❑ Round Robin - **Architecture**

```
void main()  
{ while (TRUE)  
    {  
        if ( Device_A_needs_service() )  
            { //Service device A }  
        if ( Device_A_needs_service() )  
            { //Service device A }  
        if ( Device_A_needs_service() )  
            { //Service device A }  
    }  
}
```

2.2 Basic real-time software architecture

□ Round Robin - **Pros and Cons**

- **Advantages:**

- Simple solution, but sufficient for some applications.
- Exchanging data between tasks is easy.

- **Drawbacks:**

- The worst-case latency of an external request is equal to the execution time of the entire main loop.

-

2.2 Basic real-time software architecture

❑ Round Robin - **Pros and Cons**

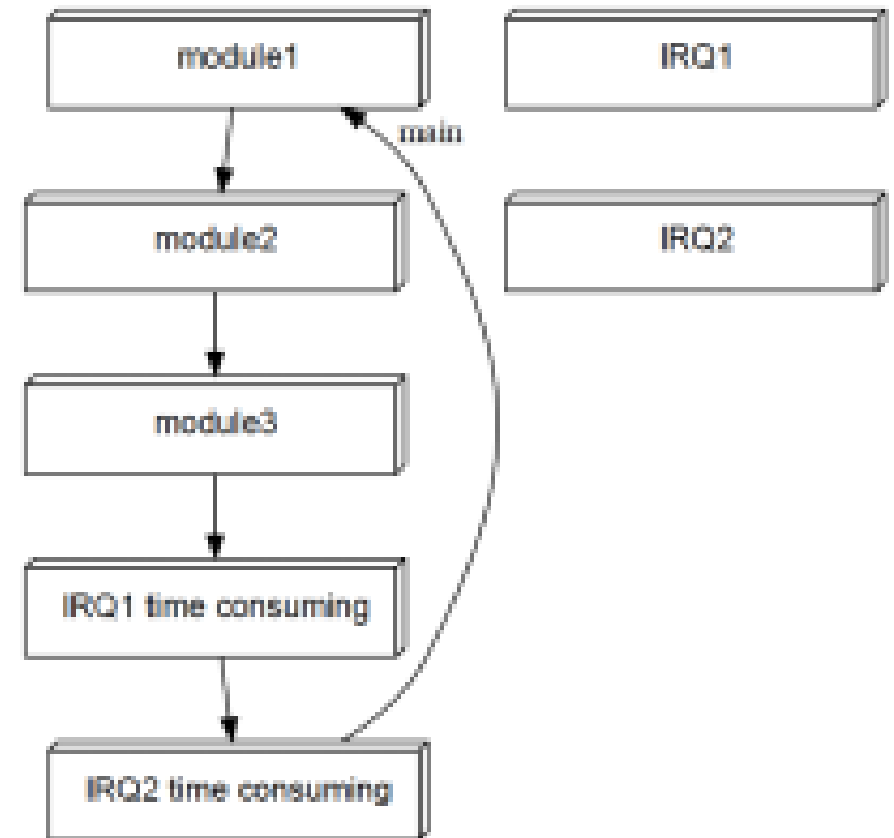
- Drawbacks:

- Architecture **fails** if any one device **requires** a **shorter response time**.
- **Implementing** additional features **can negative affect** the system's correctness , by increasing latencies over acceptable bounds.
- Architecture **is breakable** to added functionality: adding one more device to the loop may break everything.

2.2 Basic real-time software architecture

2). Round-robin with interrupts

- + Allows some control of software execution.
- + Gives more control over priorities.
- + Based on Round Robin, but interrupts deal with urgent timing requirements.
- + Interrupts **a) service hardware** and **b) set flags.**
- + Main routine checks flags and does any lower priority follow-up processing.



2.2 Basic real-time software architecture

□ Round-robin with interrupts

→ **Principles:** Tasks are invoked in round-robin fashion, but interrupt routines take care of urgent operations.

+ **A little bit more control:**

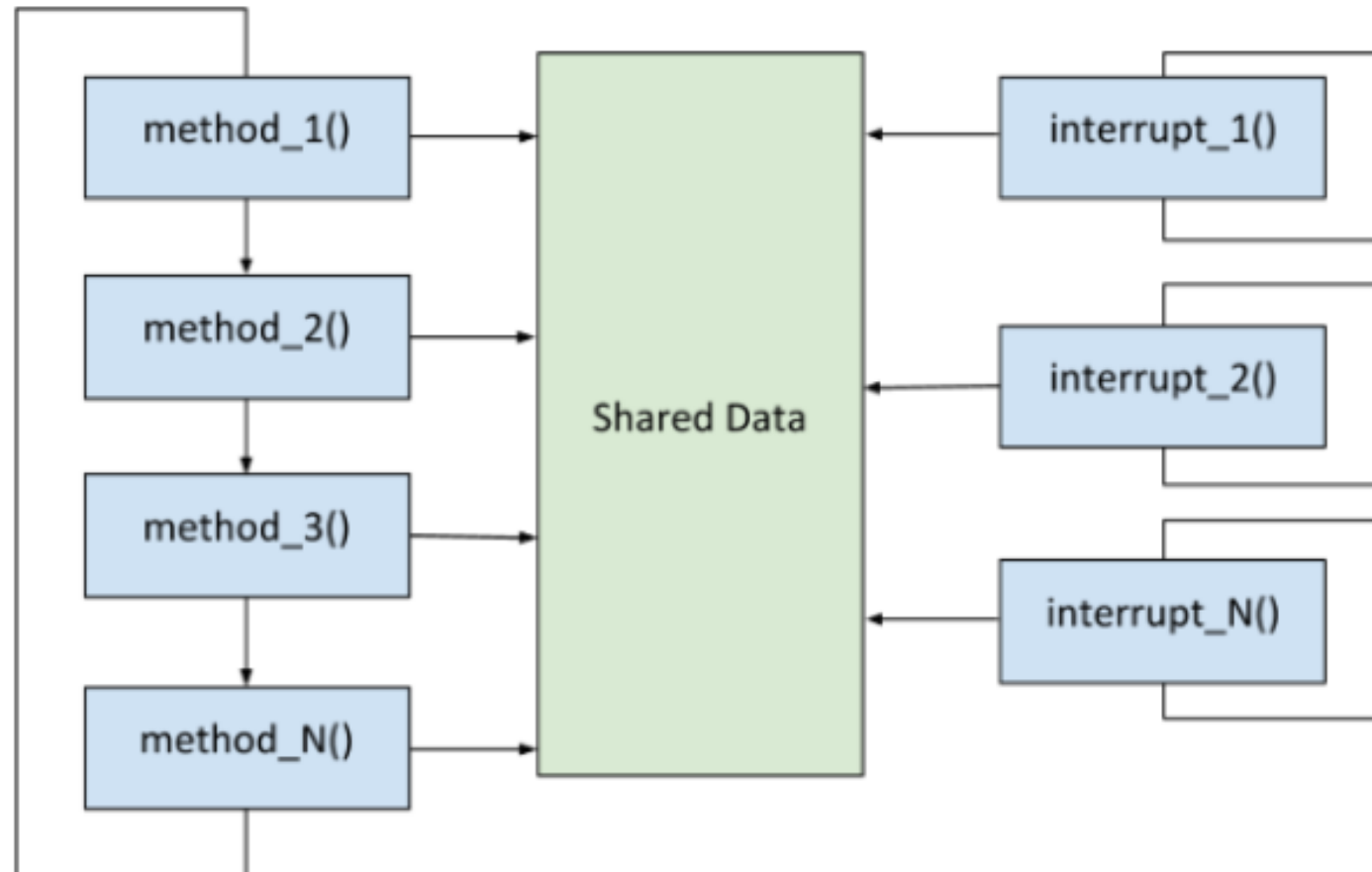
- In this architecture, **interrupt service routines (ISR)** deal with the very urgent needs of the hardware and set corresponding flags.

- Interrupt routines **set flags** to **indicate the interrupt happened main** while loop polls the status of the interrupt flags and does any follow-up processing required by a set flag.

2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG

2.2 Basic real-time software architecture

□ Round-robin with interrupts



2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG

CHƯƠNG 3: KỸ THUẬT PHẦN MỀM

2.2 Basic real-time software architecture

❑ Round-robin with interrupts

```
void interrupt Handle_device_A ()
```

```
{    //Service device A  
    F_deviceA = true;    }
```

```
void interrupt Handle_device_A ()
```

```
{    //Service device A  
    F_deviceB = true;    }
```

```
void interrupt Handle_device_A ()
```

```
{    //Service device A  
    F_deviceZ = true;    }
```

```
void main()
```

```
{ while (TRUE)
```

```
{    if (F_deviceA) { F_deviceA=false;  
        //Handle data to/from device A }
```

```
    if (F_deviceB) { F_deviceB=false;  
        //Handle data to/from device A }
```

```
    if (F_deviceC) { F_deviceC=false;  
        //Handle data to/from device A }
```

```
}
```

```
}
```

2.2 Basic real-time software architecture

□ Round-robin with interrupts - **Pros and Cons**

- **Advantages**

- Still relatively simple.
- Hardware timing requirements better met.

- **Drawbacks**

- All task code still executes at same priority.
- Maximum delay unchanged.
- Worst case response time = sum all other execution times + execution times of any other interrupts that occur.

2.2 Basic real-time software architecture

❑ Round-robin with interrupts - **Pros and Cons**

- **Possible improvements**

- Change order flags are checked (e.g., A,B,A,B,A,D)

- Improves response of A
- Increases latency of other tasks

- Move some task code to interrupt

- Decreases response time of lower priority interrupts
- May not be able to ensure lower priority interrupt code executes fast enough

2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG

Contents

- ☐ Choosing the best software architecture
- ☐ Basic real-time software architecture
- ☐ Real Time Operating System

2.3 Real Time Operating System

□ Real-Time System

- A real-time system is a computer system that **must satisfy bounded response-time constraints** or **risk severe consequences**, including failure.
- Real-time systems are defined as those systems in which the correctness of the system **depends** not only on the **logical result** of computation, but also **on the time** at which the results are produced.
- Real-time systems **respond to external events** in a **timely fashion** and the **response time is guaranteed**.

2.3 Real Time Operating System

❑ Failed System

- A failed system is a system that **cannot satisfy** one or more of the requirements stipulated in the **system requirements specification**.
- Hence, **rigorous specification** of the system **operating criteria**, including *timing constraints*, **is necessary**.

→ *Timing constraints* include:

- Finish time.
- Both start time and finish time.

2.3 Real Time Operating System

□ Degrees of “Real-Time”

- **Soft Real-Time System:** A soft real-time system is one in which performance is degraded but **not destroyed** by **failure to meet** response-time constraints (**Ex:** *audio and video delivery software, ..*).
- **Firm Real-Time System:** A firm real-time system is one in which a **few missed deadlines** will not lead to **total failure**, but **missing more than a few** may lead to **complete or catastrophic system failure**.
- **Hard Real-Time System:** A hard real-time system is one in which **failure to meet** even a **single deadline** may lead to **complete or catastrophic system failure** (**Ex:** *airplane sensor and autopilot, ..*).

2.3 Real Time Operating System

□ Real-Time Punctuality

- **Real-time punctuality** means that **every response time** has an average value, t_R , with **upper and lower bounds** of $(t_R + \epsilon_U)$ and $(t_R - \epsilon_L)$, respectively, and $(\epsilon_U, \epsilon_L \rightarrow 0+)$

$$\rightarrow t \in [-\epsilon_L, +\epsilon_U]$$

- This **response time** consists of independent **latency** components.
- **Examples:**

2.3 Real Time Operating System

- **Examples:**

- + An **elevator door** is automatically operated and it may have a capacitive safety edge for **sensing possible passengers** between the **closing door blades**.

- + Thus, the **door blades** can be **quickly re-opened** before they touch the passenger and cause discomfort or **even threaten the passenger's safety**.

- + What is the required system **response time** from **when it recognizes** that a passenger is between the closing door blades **and** starting to reopen the door?

2. CẤU TRÚC PHẦN MỀM HỆ THỐNG NHÚNG

2.3 Real Time Operating System

- Sensor ($t_{SE_min}=5ms$, $t_{SE_max}=15ms$)
- Hardware ($t_{HW_min}=1\ \mu s$, $t_{HW_max}=2\ \mu s$)
- System software ($t_{SS_min}=16\ \mu s$, $t_{SS_max}=48\ \mu s$)
- Application software ($t_{AS_min}=0.5\ \mu s$, $t_{AS_max}=0.5\ \mu s$)
- Door drive ($t_{DD_min}=300\ ms$, $t_{DD_max}=500\ ms$)
 - Minimum response time → **$t_{min} = 305\ ms$**
 - Maximum response time → **$t_{max} = 515\ ms$**